Section 04: Backend Fundamentals

Web インフラ・HTTP・Hono による API 開発

1. Web アプリケーションの「動く場所」

ここまでの学習で、プログラミングの基礎文法と TypeScript を学びました。
Web アプリケーションとして公開するためには、書いたコードを**インターネット上のどこか**で動かし続ける必要があります。

クライアントとサーバー（おさらい）

- **クライアント（フロントエンド）**：
  - ユーザーのブラウザ（Chrome, Safariなど）で動く。
  - 「見た目（UI）」と「操作感」を担当。
- **サーバー（バックエンド）**：
  - インターネット上のコンピュータで動く。
  - **「ビジネスロジック（重要な処理）」**と**「データの保存（DB）」**を担当。

2. サーバーの種類と進化

2-1. 従来型サーバー（VPS / クラウド VM）

自分で仮想的なコンピュータを借り、OS（Linuxなど）のインストールから行う方式です。

- **例**: AWS EC2, Google Compute Engine, さくら VPS
- **特徴**:
  - **自由度**: 最大。どんなソフトでも入れられる。
  - **運用負荷**: 高い。セキュリティアップデートやOSの設定を自分で行う必要がある。
  - **コスト**: アクセスがなくても月額固定費がかかることが多い。

2-2. サーバーレス / マネージドサービス

サーバーの管理をクラウド事業者に丸投げし、**コード（関数）をアップロードするだけ**で動く方式です。

- **例**: **Firebase Cloud Functions**, AWS Lambda
- **メリット**:
  - **運用ゼロ**: インフラの管理（パッチ適用など）が不要。
  - **オートスケーリング**: アクセスが急増しても自動でコンピュータが増えて対応する。
  - **従量課金**: 使った分だけ（実行時間や回数）課金される。
- **デメリット**: 起動に時間がかかる（コールドスタート）場合や、実行時間に制限がある。

2-3. エッジコンピューティング（最新トレンド）

サーバーレスをさらに進化させ、 **ユーザーに物理的に一番近い場所（世界中の拠点）** でコードを実行します。

- **例**: **Cloudflare Workers**, Deno Deploy
- **特徴**:
  - **超低レイテンシ**: 地球の裏側のサーバーを叩く必要がない。
  - **爆速の起動**: サーバーレスの弱点である「コールドスタート」がほぼゼロ。
  - Hono はこの環境で動かすことを強く意識して作られています。

3. Web の共通言語：HTTP 通信の深掘り

ブラウザとサーバーがデータをやり取りするための**共通のルール（プロトコル）**、それが **HTTP** です。

1. **Request (リクエスト)**:
    - 「このページをちょうだい」「このデータを保存して」というお願い。
2. **Response (レスポンス)**:
    - 「はい、どうぞ」「了解、保存したよ」「エラーだよ」という返事。

HTTP リクエストの構成要素

サーバーに何を送るのか、中身は主に 4 つです。

1. **Method (メソッド)**: 「何をしてほしいか」（GET, POSTなど）
2. **URL (パス)**: 「どこに対してか」（/users, /newsなど）
3. **Headers (ヘッダー)**: 「おまけ情報」（データの形式、認証情報など）
4. **Body (ボディ)**: 「中身」（保存したいデータなど）

HTTP メソッド：サーバーへの意思表示

メソッド	役割	シーン	ボディ
GET	データの**取得**	ページを開く、検索結果を見る	なし
POST	データの**作成**	会員登録、ツイート投稿	**あり**
PUT/PATCH	データの**更新**	プロフィールの変更	**あり**
DELETE	データの**削除**	投稿の削除、退会	なし

ステータスコード：サーバーからの返事

数字の 3 桁で「どうなったか」を一瞬で伝えます。

- **`2xx` (成功)**:
  - `200 OK`: 標準的な成功。
  - `201 Created`: 新しくデータを作ったよ。
- **`4xx` (クライアント（あなた）が悪い)**:
  - `400 Bad Request`: 送り方がおかしい。
  - `401 Unauthorized`: ログインしてない。
  - `403 Forbidden`: 権限がない。
  - `404 Not Found`: そんなページ/データはない。
- **`5xx` (サーバー（俺）が悪い)**:
  - `500 Internal Server Error`: サーバーのコードがバグった。

Web 共通のデータ形式：JSON

Part3でも扱いましたが、API でデータをやり取りする際、もっとも使われるのが **JSON (JavaScript Object Notation)** です。

```
{
  "name": "Shake",
  "age": 20,
  "skills": ["TypeScript", "Hono"],
  "isValid": true
}
```

- **なぜ JSON か？**:
  - **読みやすい**: 人間が見ても構造がすぐわかる。
  - **互換性**: 言語を問わず（Python, Java, Go...）扱える。
  - **JS/TS と相性抜群**: そのままオブジェクトとして扱える。

4. なぜサーバーで処理するのか？（ビジネスロジック）

「フロントエンド（ブラウザ）で全部処理すればいいじゃん」と思うかもしれません。
しかし、バックエンドには**絶対に譲れない役割**があります。

1. データの永続化

ブラウザを閉じても、データは消えてはいけません。データベースに保存するのはサーバーの仕事です。

2. セキュリティと権限

「他人の投稿を消せるか」「お金を振り込めるか」といったチェックは、ユーザーの手が届くブラウザ上で行うと**改ざんのリスク**があります。サーバー上で厳格にチェックします。

3. 重い処理の肩代わり

スマホなどの貧弱な端末ではなく、強力なサーバーで計算を行います。

5. Web フレームワーク Hono

Hono：世界を席巻する日本発のバックエンド・フレームワーク

Hono（ほのぉ、と発音）は、Cloudflare Workers や Firebase など、あらゆる環境で高速に動くように設計されています。

- **Web 標準**: 独自ルールを覚える必要が少なく、`Request` / `Response` をそのまま扱う。
- **型安全**: TypeScript との相性が最高。VS Code の補完が効きまくる。
- **軽量**: 余計な機能がないため、起動がとにかく速い。

核心オブジェクト： `c` (Context)

Hono のハンドラの引数にある `c` は、**「この一回の通信に必要なすべてが入った便利なカバン」** です。

```
app.get("/hello", (c) => {
  const data = c.req.json();
  // レスポンスを JSON で返す
  return c.json({ message: "Hello!", data: data });
});
```

- `c.req`：「何が届いたか？」を知る。
- `c.json()` / `c.text()`：「何を返すか？」を決める。

6. クライアントからデータを送る 3 つのパターンと、その受け取り方

A. データの送り方

1. パスパラメータ（URL の一部にする）

特定のデータを指すときに使います。

- URL: `/users/shake`
- コード: `app.get("/users/:username", (c) => ...)`
- Honoでの取得: `const name = c.req.param("username")`

使われている場所：ユーザー詳細画面、記事詳細画面など

具体的なURL例

- http://github.com/Shakenokirimi12
- http://github.com/Shakenokirimi12/sosho-2026
  それぞれ、`Shakenokirimi12` というユーザー名や `sosho-2026` というリポジトリ名がパスパラメータとして扱われます。

2. クエリパラメータ（?以降に付ける）

検索条件やフィルタリングに使います。

- URL: `/search?q=hono&sort=new`
- Honoでの取得: `c.req.query("q")` (結果："hono")

> ミニクイズ：ここでc.req.query("sort")とした場合はどうなるでしょう？

具体的なURL例

- https://google.com/search?q=hono&sort=new
  `q` に"hono"、 `sort` に"new"がそれぞれクエリパラメータとして扱われます。
  多くの検索サービスでは、検索ワードがクエリパラメータに格納されてサーバーに送信されています。

3. JSON ボディ（通信の中身に入れる）

複雑なデータ（会員登録情報など）を送るときに使います。
Bodyが使用できる、POST, PUT, PATCHメソッドで利用されます。

- 取得: `const body = await c.req.json()`

例えば、ECサイトで「ある商品」を「1つ」カートに入れるとき、

- **データの送り先**: `/cart/add`
- **データの内容 (Body)**:

```
{
  "productId": "SHAKE-777",
  "quantity": 1
}
```

このように、 **「何を（商品ID）」「どれだけ（数量）」** といった具体的な情報を、URLやクエリパラメータではなく **通信の内容（Body）** に詰め込んで送ります。

B. Hono のルーティングと「データの受け取り」

1. 基本のパスとメソッド

Hono は **「どのメゾットで」「どのパスに」** 来たかを見て、動かす関数を決めます。

```
// GET /posts (記事一覧を取得する)
app.get("/posts", (c) => {
  return c.text("記事一覧です");
});

// POST /posts (新しい記事を作成する)
app.post("/posts", (c) => {
  return c.text("記事を作成しました");
});
```

同じ `/posts` というパスでも、**メソッドが違えば別々の処理**として定義できます。

2. パスパラメータ (`c.req.param`)

URL の一部を「変数」として扱いたい時に使います。特定のデータを指すときに便利です。

```
app.get("/users/:username", (c) => {
  const name = c.req.param("username");
  return c.text(`こんにちは、${name}さん!`);
});
```

- `:username`: この部分が変数になります。
- **取得**: `c.req.param("変数名")` で中身を取り出します。
- このコードで受理されるパス
  - `/users/shake`
  - `/users/kirimi`
- このコードで受理されないパス
  - `/users`

3. クエリパラメータ (`c.req.query`)

URL の末尾に `?q=hono` のように付ける、検索やフィルタ用の「オプション」です。

```
// /search?q=hono&limit=10
app.get("/search", (c) => {
  const query = c.req.query("q");
  const limit = c.req.query("limit");

  return c.json({ word: query, max: limit });
});
```

特徴

- **常に文字列**: 数値として使いたい場合は `Number(limit)` のように変換が必要です。
- **省略可能**: 指定されない場合に備えて、デフォルト値を決めておくと安全です。

4. 定義する「順番」がとても大事

Hono は、コードの **上から順番に** パスが一致するかチェックします。

```
// A: パスパラメータ(広い)
app.get("/posts/:id", (c) => c.text("記事詳細"));

// B: 通常のパス(具体的)
app.get("/posts/popular", (c) => c.text("人気記事"));
```

この順序だと、`/posts/popular` にアクセスしても **A に吸い込まれてしまいます**
（`:id` が `"popular"` だと解釈されるため）。
「具体的なパス」は上に、「広いパス（パラメータ）」は下にするのが鉄則です。

💡 ミニクイズ：どっちが動く？

```
app.get("/user/:id", (c) => c.text("A"));
app.post("/user/new", (c) => c.text("B"));
app.get("/user/new", (c) => c.text("C"));
```

1. `GET` で `/user/new` にアクセスした時：？
2. `POST` で `/user/new` にアクセスした時：？

```
app.get("/user/:id", (c) => c.text("A"));
app.post("/user/new", (c) => c.text("B"));
app.get("/user/new", (c) => c.text("C"));
```

1. `GET` で `/user/new` にアクセスした時
2. `POST` で `/user/new` にアクセスした時

答え

1. **A** が動きます。
   - `GET /user/new` は、1行目の `GET /user/:id` （`:id` が `"new"`）に一致してしまうため、3行目まで到達しません。
2. **B** が動きます。
   - メソッドが `POST` なので、1行目（GET）には一致せず、2行目の POST 処理が正しく動きます。

5. データの「型」を決める (Interface)

バックエンドでは、届いたデータ（特に JSON Body）がどんな形かを知るために `interface` を定義します。

```
interface CreateUserRequest {
  name: string;
  age: number;
}

app.post("/users", async (c) => {
  // <CreateUserRequest> と書くことで、body に型がつきます
  const body = await c.req.json<CreateUserRequest>();
  return c.json({ status: "ok", received: body.name });
});
```

> **注意**: Interface はあくまで開発者向け、VS Code の補完を効かせるための「お約束」です。悪意あるユーザーが不正なデータを送ってきた場合、これだけでは防げません（次Partの「バリデーション」で解決します）。

【深掘り】この数行で何が起きているのか？ (1/4)

```
interface CreateUserRequest {
  name: string;
  age: number;
}
```

- **何のため？**: クライアントから届くデータの「形」を TypeScript に教えます。
- **「型（Type）」のメリット**:
  - `body.naem` （タイポ）などを書こうとすると VS Code が赤線を出してくれます。
  - チーム開発で「どんなデータを送ればいいか」のドキュメント代わりになります。

【深掘り】この数行で何が起きているのか？ (2/4)

```
app.post("/users", async (c) => { ... });
```

- `app.post`：「POST メソッド（作成）」でリクエストが来た時だけ動く、というルール。
- `async (c) => { ... }`：
  - `async`：通信やデータベース操作など、「待ち時間」が発生する処理を含む関数に付けます。
  - `(c)`：先ほど学んだ「カバン（Context）」を受け取る口です。

【深掘り】この数行で何が起きているのか？ (3/4)

```
const body = await c.req.json<CreateUserRequest>();
```

- `await`：データの読み込みが終わるまで、次の行に行かずに「待機」します。これがないと、空っぽのまま処理が進んでしまいます。
- `<CreateUserRequest>` **(ジェネリクス)**:
  - Hono に「これから読み込む JSON は、さっき決めた `CreateUserRequest` の形をしているよ！」と強く教える魔法です。
  - これにより、`body.name` と打った時に**自動補完**が効くようになります。

【深掘り】この数行で何が起きているのか？ (4/4)

```
return c.json({ status: "ok" });
```

- `c.json(...)`:
  - JavaScript のオブジェクトを、自動的に HTTP 通信用の **JSON 形式**に変換して返します。
  - ヘッダー（`Content-Type: application/json`）も自動で付けてくれる賢い機能です。
- `return`: これで「リクエストへの返信」が完了し、この一回の通信処理が終了します。

7. 実践：Hono × Cloudflare Workers で API を構築する

初心者に最も優しく、かつ強力なプラットフォームである **Cloudflare Workers** を使って、実際に動く API を作ってみましょう。

Cloudflare Workers の魅力

- **クレジットカード不要**: 1日10万リクエストまで無料で使えます。
- **爆速セットアップ**: 1分で Hello World がインターネットに公開できます。
- **デプロイが簡単**: コマンド一つで世界中のサーバーに配信されます。

Cloudflare Workers とは？

「あなたのコードを世界中の爆速サーバーで動かす」 仕組みです。

- **サーバーの管理がゼロ**: 自分でサーバーを立てたり、OS の設定をしたりする必要はありません。
- **エッジ（Edge）で動く**: 東京、大阪、ニューヨークなど、世界 300 以上の拠点にあるサーバーのうち、**ユーザーに一番近い場所** でプログラムが実行されます。
- **一瞬で起動**: 他のサービス（Lambda 等）で起きがちな「起動が遅い（コンコードスタート）」という問題がほぼありません。

Cloudflare D1 とは？

「世界中に分散された、設定不要の SQL データベース」 です。

- **D1 = SQLite**: 世界で最も使われているデータベースエンジン「SQLite」がベースになっています。
- **Workers との相性が最高**: 同じ Cloudflare のネットワーク内にあるため、通信の遅延が極限まで抑えられています。
- **サーバーレス**: データの量に合わせて自動でスケールし、使った分だけ（無料枠内なら 0 円）の料金で済みます。

ステップ 0: 事前準備（アカウント作成とログイン）

Cloudflare のサービスを使うために、まずは自分のアカウントを作成し、PC からアクセスできるようにします。

1. **Cloudflare 公式サイト** にアクセスし、メールアドレスでアカウントを作成します。
    - 無料枠（Free Plan）で十分ですので、クレジットカードの登録は不要な場合がほとんどです。
2. ターミナルで以下のコマンドを入力し、自分の PC と Cloudflare を連携（ログイン）させます。

```
npx wrangler login
```

- コマンドを打つとブラウザが開くので、**「Allow（許可）」** をクリックすれば完了です！

ステップ 1: プロジェクトの作成

ターミナルで以下のコマンドを入力します。

```
# Hono のテンプレートからプロジェクトを作成
npm create hono@latest my-api
```

- **Target directory**: `my-api` (そのまま Enter)
- **Template**: `cloudflare-workers` を選択
- **Install dependencies**: `yes`

作成されたディレクトリに移動します。

```
cd my-api
```

ステップ 2: データベース (D1) の作成

Cloudflare のサーバー上に、あなた専用のデータベースを作成します。

```
npx wrangler d1 create my-db
```

実行後、ターミナルに以下のようなデータベースの設定が表示されます。プロジェクト内の `wrangler.jsonc` に、以下のように `d1_databases` の設定を追記して保存してください。

```
{
  // ... 他の設定
  "d1_databases": [
    {
      "binding": "DB", // コード内で使う名前
      "database_name": "my-db",
      "database_id": "xxxx-xxxx-xxxx", // データベースのID
    },
  ],
}
```

ステップ 3: テーブルの作成 (Schema)

データベースに「ユーザー情報を入れる箱（テーブル）」を作ります。
プロジェクト直下に `schema.sql` というファイルを作り、以下の SQL を書き込みます。

```
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT UNIQUE
);
```

この SQL を実行して、データベースに反映させます（ローカル環境用）。

```
npx wrangler d1 execute my-db --local --file=./schema.sql
```

ステップ 4: API を実装する (`src/index.ts`)

`src/index.ts` を開き、中身を以下のように書き換えます。

```
import { Hono } from "hono";

// 型定義:環境変数に DB があることを教える
type Bindings = {
  DB: D1Database;
};

const app = new Hono<{ Bindings: Bindings }>(); // 環境変数にDBがあることを、Honoの初期化時に伝える

// 1. ユーザー一覧の取得 (SELECT)
app.get("/users", async (c) => {
  const { results } = await c.env.DB.prepare("SELECT * FROM users").all();
  return c.json(results);
});

// 2. ユーザーの追加 (INSERT)
app.post("/users", async (c) => {
  const { name, email } = await c.req.json();
  await c.env.DB.prepare("INSERT INTO users (name, email) VALUES (?, ?)").bind(name, email).run();
  return c.json({ message: "作成しました" }, 201);
});

export default app;
```

ステップ 5: 動作確認とデプロイ

ローカルでテスト

```
npm run dev
```

本番へ公開 (デプロイ)

```
# 1. 本番のデータベースにもテーブルを作る
npx wrangler d1 execute my-db --remote --file=./schema.sql

# 2. 公開!
npm run deploy
```

8. まとめ

1. **インフラ**: サーバー管理不要の「サーバーレス」と、設定不要の「D1（SQL）」の組み合わせが最強。
2. **HTTP**: Web の基本ルール。メソッドを使い分けて、意図を伝える。
3. **JSON**: 共通言語。フロントとバックを結ぶ架け橋。
4. **Hono**: どんな環境でも、シンプルかつ型安全にバックエンドを組める。
5. **D1 (SQL)**: 世界中に分散された、超高速でスケーラブルなデータベース。

おめでとうございます！これであなたは、モダンなバックエンド開発の全工程（開発・DB構築・公開）を体験しました。

次は、届いたデータの不備を自動で弾く「バリデーション（Zod）」と、より複雑なデータ操作に挑みます。